المدرسة الوطنية للعلوم التطبيقية -أكادير
ⵜⵉⵎⵙⵙⵓⵜ ⵏ ⵜⵓⵙⵙⵏⵉⵡⵉⵏ ⵜⵉⵖⵓⵔⵉⵡⵉⵏ - ⴰⴳⴰⴷⵉⵔ.
ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES - AGADIR

Pricing des options

Réalisé par :

EL IDRISSI Salma

Encadré par :

Pr. B. EL ASRI

Filière : Finance et Ingénierie Décisionnelle

Année universitaire : 2025–2026 **Déposé le : 20 Janvier 2026**

Table des matières

Introduction générale **6**

1 Rappels théoriques sur les options **8**
1.1 Définitions et typologie 8
1.2 Concepts clés 8
1.2.1 Prix d'exercice (Strike Price) 8
1.2.2 Maturité (Time to Expiry) 8
1.2.3 Valeur intrinsèque et valeur temporelle 9
1.2.4 Moneyness 9
1.3 Options européennes vs américaines 9
1.4 Hypothèses de marché et pricing risque-neutre 10
1.4.1 Hypothèses traditionnelles du modèle de Black-Scholes 10
1.4.2 Principe d'absence d'arbitrage 10
1.4.3 Évaluation en probabilité risque-neutre 10
1.4.4 Extension avec dividendes 11

2 Modèles stochastiques pour le sous-jacent **12**
2.1 Mouvement Brownien Géométrique (GBM) et modèle de Black-Scholes . . 12
2.1.1 Définition et équation différentielle stochastique 12
2.1.2 Solution analytique 12
2.1.3 Modèle de Black-Scholes-Merton 13
2.1.4 Simulation numérique 13
2.1.5 Limites du modèle GBM 13
2.2 Modèle de Heston (volatilité stochastique) 14
2.2.1 Motivation et formulation 14
2.2.2 Interprétation des paramètres 14
2.2.3 Avantages du modèle 14
2.2.4 Simulation numérique 14
2.3 Modèle de Merton (jump-diffusion) 15
2.3.1 Introduction des sauts 15
2.3.2 Formulation mathématique 15

2.3.3 Solution et simulation . . . 15
2.3.4 Avantages et applications . . . 16
2.4 Modèles avec dividende . . . 16
2.4.1 Dividendes continus . . . 16
2.4.2 Dividendes discrets . . . 16
2.4.3 Impact sur le pricing d'options . . . 16
2.5 Comparaison des modèles . . . 17

3 Méthodes de pricing numérique 18
3.1 Méthode de Monte Carlo : principe, convergence et intervalle de confiance . 18
3.1.1 Principe fondamental . . . 18
3.1.2 Estimateur Monte Carlo . . . 18
3.1.3 Convergence et erreur . . . 19
3.1.4 Intervalle de confiance . . . 19
3.1.5 Avantages et inconvénients . . . 19
3.2 Simulation des trajectoires : discrétisation et génération de variables . . . 20
3.2.1 Discrétisation temporelle . . . 20
3.2.2 Schémas de discrétisation . . . 20
3.2.3 Génération de variables aléatoires . . . 20
3.2.4 Implémentation dans notre code . . . 20
3.3 Pricing d'options européennes par Monte Carlo . . . 21
3.3.1 Validation par comparaison avec Black-Scholes . . . 21
3.3.2 Extensions aux modèles complexes . . . 21
3.4 Pricing d'options américaines par la méthode de Longstaff-Schwartz . . . 21
3.4.1 Problématique de l'exercice anticipé . . . 21
3.4.2 Principe de la méthode LSM . . . 21
3.4.3 Algorithme LSM . . . 22
3.4.4 Fonctions de base pour la régression . . . 22
3.4.5 Implémentation dans notre application . . . 22
3.4.6 Validation de l'implémentation . . . 22
3.4.7 Complexité et performance . . . 23

4 Techniques d'amélioration de la simulation 24
4.1 Variables antithétiques . . . 24
4.1.1 Principe . . . 24
4.1.2 Algorithme . . . 24
4.1.3 Analyse de la réduction de variance . . . 25
4.1.4 Implémentation dans notre code . . . 25
4.1.5 Avantages et limites . . . 25
4.2 Variables de contrôle . . . 25

4.2.1 Principe . . . 25
4.2.2 Choix de la variable de contrôle . . . 26
4.2.3 Calcul du coefficient optimal . . . 26
4.2.4 Algorithme . . . 26
4.2.5 Implémentation dans notre code . . . 26
4.2.6 Avantages et limites . . . 26
4.3 Comparaison des gains en efficacité . . . 27
4.3.1 Mesure de l'efficacité . . . 27
4.3.2 Résultats empiriques . . . 27
4.3.3 Exemple numérique . . . 27
4.3.4 Choix de la méthode . . . 28
4.4 Autres méthodes non implémentées . . . 28
4.4.1 Échantillonnage préférentiel (Importance Sampling) . . . 28
4.4.2 Stratification . . . 28
4.5 Résumé . . . 28

5 Implémentation et application interactive 29
5.1 Architecture du code (classes et fonctions) . . . 29
5.1.1 Structure générale . . . 29
5.1.2 Les classes principales . . . 29
5.1.3 Design patterns utilisés . . . 30
5.2 Interface utilisateur avec Streamlit . . . 31
5.2.1 Pourquoi Streamlit ? . . . 31
5.2.2 Structure de l'interface . . . 31
5.2.3 Design et style . . . 32
5.3 Visualisations et analyse interactive . . . 32
5.3.1 Trajectoires simulées . . . 32
5.3.2 Analyse de convergence Monte Carlo . . . 32
5.3.3 Distribution des prix finaux . . . 32
5.3.4 Frontière d'exercice pour les options américaines . . . 33
5.3.5 Comparaison des modèles . . . 33
5.4 Validation et tests . . . 33
5.4.1 Validation numérique . . . 33
5.4.2 Tests de performance . . . 34
5.4.3 Tests d'interface . . . 34
5.4.4 Exemples de validation . . . 34
5.4.5 Limitations identifiées . . . 35
5.5 Synthèse . . . 35

6 Limites et perspectives **36**
6.1 Limites théoriques des modèles . . . 36
6.1.1 Limites du modèle GBM . . . 36
6.1.2 Limites du modèle de Heston . . . 36
6.1.3 Limites du modèle de Merton . . . 37
6.1.4 Limites des méthodes de pricing numérique . . . 37
6.2 Limites pratiques de l'implémentation . . . 37
6.2.1 Performances et scalabilité . . . 37
6.2.2 Interface utilisateur . . . 38
6.2.3 Validation et robustesse . . . 38
6.2.4 Manque de fonctionnalités avancées . . . 38
6.3 Améliorations possibles . . . 39
6.3.1 Améliorations théoriques et numériques . . . 39
6.3.2 Améliorations d'implémentation . . . 39
6.3.3 Extensions de l'application . . . 40
6.3.4 Perspectives pédagogiques . . . 41

7 Conclusion **42**
7.1 Synthèse des travaux . . . 42
7.1.1 Fondements théoriques . . . 42
7.1.2 Modèles stochastiques . . . 42
7.1.3 Méthodes numériques . . . 43
7.1.4 Implémentation logicielle . . . 43
7.1.5 Validation et analyse . . . 43
7.2 Importance de la méthode Monte Carlo pour options complexes . . . 43
7.2.1 Flexibilité . . . 43
7.2.2 Dimensionnalité . . . 44
7.2.3 Options américaines . . . 44
7.2.4 Limites et améliorations . . . 44
7.3 Apports de l'interface Streamlit . . . 44
7.3.1 Développement rapide . . . 44
7.3.2 Visualisations interactives . . . 45
7.3.3 Gestion d'état . . . 45
7.3.4 Limites de Streamlit . . . 45
7.4 Conclusion générale . . . 45

Introduction générale

Le Pricing d'Options : Entre Théorie Mathématique et Application Numérique Interactive

Les marchés financiers contemporains sont caractérisés par une complexité croissante, où les instruments dérivés occupent une place centrale dans la gestion des risques et les stratégies d'investissement. Parmi ces instruments, les options financières, qui confèrent à leur détenteur le droit - mais non l'obligation - d'acheter ou de vendre un actif sous-jacent à un prix et à une échéance déterminés, constituent des outils de couverture et de spéculation essentiels. Leur évaluation précise, communément appelée *pricing*, représente un enjeu fondamental tant pour les praticiens de la finance que pour les chercheurs en mathématiques financières.

Historiquement, le modèle de Black-Scholes-Merton (1973) a marqué une révolution en proposant une formule analytique fermée pour le prix des options européennes, sous un ensemble d'hypothèses simplificatrices incluant notamment la constance de la volatilité. Ce modèle fondateur, récompensé par le Prix Nobel d'Économie en 1997, a établi les bases de la théorie moderne du pricing d'options. Cependant, les observations empiriques des marchés - avec leurs crises, leurs sauts de prix et leur volatilité changeante - ont rapidement révélé les limites de ce modèle pionnier. Les praticiens constatent quotidiennement la présence d'un *smile* ou d'un *skew* de volatilité, phénomène que le modèle à volatilité constante ne peut expliquer.

Pour remédier à ces limitations, des modèles plus sophistiqués ont été développés. Le modèle de Heston (1993) introduit une volatilité stochastique, permettant de capturer la dynamique de la volatilité et ses corrélations avec le prix du sous-jacent. Le modèle de Merton (1976) incorpore des sauts dans la dynamique des prix, rendant compte des mouvements brutaux observés sur les marchés. Parallèlement, l'intégration de dividendes dans les modèles permet de mieux coller aux caractéristiques des actions versant des revenus réguliers.

Sur le plan numérique, l'avènement de méthodes de calcul puissantes a permis de s'affranchir des limitations des solutions analytiques. La méthode de Monte Carlo, initialement développée dans le contexte du projet Manhattan avant d'être transposée en finance, offre une flexibilité incomparable pour le pricing d'options complexes. Sa conver-

gence relativement lente, de l'ordre de $\mathcal{O}(1/\sqrt{N})$, est compensée par des techniques de réduction de variance ingénieuses telles que les variables antithétiques ou les variables de contrôle. Pour les options américaines, caractérisées par la possibilité d'exercice anticipé, l'algorithme de Longstaff-Schwartz (2001) constitue une avancée majeure en combinant simulation Monte Carlo et régression pour déterminer la frontière d'exercice optimale.

Ces avancées théoriques et numériques restent cependant souvent cantonnées à des cercles académiques ou à des systèmes propriétaires peu accessibles. C'est dans ce contexte que notre travail s'inscrit : nous proposons **QuantOption Pro**, une application interactive de pricing d'options développée en Python, qui vise à combler le fossé entre théorie complexe et application pédagogique. Cette plateforme intègre les modèles stochastiques avancés (GBM, Heston, Merton), implémente les méthodes numériques modernes (Monte Carlo avec réduction de variance, Longstaff-Schwartz), et offre une interface intuitive via Streamlit permettant une exploration interactive des concepts.

L'objectif de ce mémoire est triple :

1. Présenter les fondements théoriques du pricing d'options sous différents modèles stochastiques ;
2. Détailler l'implémentation numérique et l'architecture logicielle de notre application ;
3. Analyser les résultats obtenus et leurs implications pratiques.

Chapitre 1

Rappels théoriques sur les options

1.1 Définitions et typologie

Une option est un contrat financier qui confère à son détenteur le droit, mais non l'obligation, d'acheter ou de vendre un actif sous-jacent à un prix prédéterminé (le prix d'exercice ou *strike*) à une date future spécifiée (la date d'échéance ou *maturité*). L'acheteur de l'option paie une prime au vendeur pour obtenir ce droit.

On distingue deux grandes catégories d'options :

— **Options d'achat (Call)** : donnent le droit d'acheter l'actif sous-jacent au prix d'exercice.

— **Options de vente (Put)** : donnent le droit de vendre l'actif sous-jacent au prix d'exercice.

Selon les modalités d'exercice, on classe les options en :

— **Options européennes** : ne peuvent être exercées qu'à la date d'échéance.

— **Options américaines** : peuvent être exercées à tout moment jusqu'à la date d'échéation.

1.2 Concepts clés

1.2.1 Prix d'exercice (Strike Price)

Le prix d'exercice, noté K, est le prix auquel le détenteur de l'option peut acheter (call) ou vendre (put) l'actif sous-jacent. C'est un paramètre fixé lors de la création du contrat.

1.2.2 Maturité (Time to Expiry)

La maturité, notée T, représente le temps restant jusqu'à la date d'échéance de l'option. Elle influence directement la valeur de l'option : plus la maturité est longue, plus l'option

a de temps pour devenir rentable.

1.2.3 Valeur intrinsèque et valeur temporelle

La valeur d'une option se décompose en deux parties :

— **Valeur intrinsèque** : gain immédiat si l'option était exercée maintenant.

$$\text{Call} : \max(S - K, 0)$$
$$\text{Put} : \max(K - S, 0)$$

où S est le prix courant du sous-jacent.

— **Valeur temporelle** : valeur supplémentaire liée à l'incertitude future. Elle dépend de la volatilité, du temps restant et des taux d'intérêt.

La valeur totale d'une option est la somme de sa valeur intrinsèque et de sa valeur temporelle.

1.2.4 Moneyness

Le *moneyness* caractérise la position du prix du sous-jacent par rapport au prix d'exercice :

— **In the Money (ITM)** : option avec valeur intrinsèque positive

— **At the Money (ATM)** : prix du sous-jacent égal au prix d'exercice

— **Out of the Money (OTM)** : option sans valeur intrinsèque

1.3 Options européennes vs américaines

La distinction fondamentale entre options européennes et américaines réside dans la flexibilité d'exercice :

Caractéristique	**Option européenne**	**Option américaine**
Exercice	À la maturité seulement	À tout moment jusqu'à maturité
Prix	Généralement moins cher	Généralement plus cher
Complexité du pricing	Plus simple	Plus complexe
Formule fermée	Black-Scholes disponible	Pas de formule fermée générale

TABLE 1.1 – Comparaison options européennes et américaines

Pour un call sur une action ne versant pas de dividende, il n'est jamais optimal d'exercer l'option américaine avant l'échéance. Dans ce cas, le prix de l'option américaine est égal au prix de l'option européenne correspondante. Pour les puts et les calls avec dividendes, l'exercice anticipé peut être optimal.

1.4 Hypothèses de marché et pricing risque-neutre

1.4.1 Hypothèses traditionnelles du modèle de Black-Scholes

Le modèle de Black-Scholes repose sur plusieurs hypothèses fondamentales :

1. Le sous-jacent suit un mouvement brownien géométrique avec dérive et volatilité constants.
2. Pas de dividendes pendant la vie de l'option (version de base).
3. Pas de frais de transaction ou d'impôts.
4. Pas de restrictions sur les ventes à découvert.
5. Le trading est continu.
6. Le taux d'intérêt sans risque est constant et connu.
7. Les options sont de style européen.

1.4.2 Principe d'absence d'arbitrage

L'absence d'opportunités d'arbitrage est un principe fondamental en finance qui stipule qu'il n'est pas possible de réaliser un profit sans risque avec un investissement initial nul. Ce principe conduit à des relations de parité entre options, comme la relation de parité call-put :

$$C - P = Se^{-qT} - Ke^{-rT}$$

où C est le prix du call, P le prix du put, S le prix du sous-jacent, K le prix d'exercice, r le taux sans risque, q le taux de dividende, et T la maturité.

1.4.3 Évaluation en probabilité risque-neutre

Le pricing risque-neutre est une technique fondamentale qui permet de valoriser les options en actualisant les flux futurs espérés au taux sans risque, sous une probabilité dite "risque-neutre" où les actifs rapportent en moyenne le taux sans risque.

— **Probabilité risque-neutre** : probabilité artificielle sous laquelle le prix actualisé de tout actif est une martingale.

— **Formule générale de pricing** :

$$V_0 = e^{-rT} E^Q[V_T]$$

où E^Q désigne l'espérance sous la probabilité risque-neutre Q, et V_T est la valeur de l'option à l'échéance.

— **Avantage** : permet de s'affranchir de l'aversion au risque des investisseurs en incorporant celle-ci dans le changement de probabilité.

1.4.4 Extension avec dividendes

Dans la pratique, de nombreuses actions versent des dividendes. Le modèle de Black-Scholes a été étendu par Merton pour prendre en compte un taux de dividende continu q. Dans ce cas, la dérive du sous-jacent sous la probabilité risque-neutre devient $r - q$ au lieu de r.

Ces concepts théoriques constituent la base sur laquelle reposent les modèles plus sophistiqués et les méthodes numériques que nous aborderons dans les chapitres suivants. Ils permettent de comprendre les mécanismes fondamentaux de valorisation des options avant d'introduire des complexités additionnelles telles que la volatilité stochastique ou les sauts de prix.

Chapitre 2

Modèles stochastiques pour le sous-jacent

Les modèles stochastiques sont au cœur de la modélisation des actifs financiers. Ils visent à capturer la dynamique du prix des actifs sous-jacents, en tenant compte de leur caractère aléatoire. Ce chapitre présente les principaux modèles utilisés dans notre application, depuis le classique Mouvement Brownien Géométrique jusqu'aux modèles plus sophistiqués de Heston et Merton.

2.1 Mouvement Brownien Géométrique (GBM) et modèle de Black-Scholes

2.1.1 Définition et équation différentielle stochastique

Le Mouvement Brownien Géométrique (GBM) est le modèle le plus fondamental en finance quantitative. Il postule que le prix S_t d'un actif suit l'équation différentielle stochastique (EDS) :

$$dS_t = \mu S_t dt + \sigma S_t dW_t \tag{2.1}$$

où :
— μ est le taux de dérive (drift) attendu
— σ est la volatilité (constante)
— W_t est un mouvement brownien standard
— $dW_t \sim \mathcal{N}(0, dt)$

2.1.2 Solution analytique

L'équation (2.1) admet une solution analytique explicite :

$$S_t = S_0 \exp\left[\left(\mu - \frac{\sigma^2}{2}\right) t + \sigma W_t\right] \tag{2.2}$$

Cette solution montre que le prix suit une distribution log-normale, c'est-à-dire que $\ln(S_t/S_0)$ suit une distribution normale.

2.1.3 Modèle de Black-Scholes-Merton

Dans le cadre du pricing d'options, on travaille sous la probabilité risque-neutre Q. Sous cette probabilité, le drift μ est remplacé par le taux sans risque r (et éventuellement un taux de dividende q). L'EDS devient :

$$dS_t = (r - q)S_t dt + \sigma S_t dW_t^Q \tag{2.3}$$

où W_t^Q est un mouvement brownien standard sous Q.

2.1.4 Simulation numérique

Pour simuler des trajectoires du GBM, on utilise une discrétisation de l'équation (2.2). Sur un intervalle de temps Δt, l'évolution du prix est donnée par :

$$S_{t+\Delta t} = S_t \exp\left[\left(r - q - \frac{\sigma^2}{2}\right) \Delta t + \sigma\sqrt{\Delta t} Z\right] \tag{2.4}$$

où $Z \sim \mathcal{N}(0, 1)$.

Dans notre implémentation (classe `StochasticModels.geometric_brownian_motion`), nous utilisons cette formule pour générer des trajectoires complètes sur N pas de temps.

2.1.5 Limites du modèle GBM

Bien que le GBM soit simple et analytiquement traitable, il présente plusieurs limitations :

- **Volatilité constante** : ne capture pas le smile/skew de volatilité observé sur les marchés
- **Absence de sauts** : ne modélise pas les mouvements brusques des prix
- **Normalité des rendements** : les rendements réels présentent souvent des queues épaisses (excès de kurtosis)

2.2 Modèle de Heston (volatilité stochastique)

2.2.1 Motivation et formulation

Pour pallier la limitation de la volatilité constante, Steven Heston a proposé en 1993 un modèle où la variance (carré de la volatilité) suit elle-même un processus stochastique. Le modèle de Heston est décrit par le système d'EDS suivant :

$$dS_t = (r - q)S_t dt + \sqrt{v_t} S_t dW_t^S \tag{2.5}$$

$$dv_t = \kappa(\theta - v_t)dt + \sigma_v \sqrt{v_t} dW_t^v \tag{2.6}$$

$$dW_t^S dW_t^v = \rho dt \tag{2.7}$$

où :

— v_t est la variance instantanée (volatilité au carré)
— $\kappa > 0$ est la vitesse de retour à la moyenne
— $\theta > 0$ est le niveau de variance de long terme
— $\sigma_v > 0$ est la "volatilité de la volatilité"
— $\rho \in [-1, 1]$ est la corrélation entre les deux browniens

2.2.2 Interprétation des paramètres

— **Vitesse de retour** κ : contrôle la rapidité avec laquelle la variance revient vers son niveau de long terme θ. Un κ élevé signifie un retour rapide.
— **Niveau de long terme** θ : niveau moyen autour duquel la variance fluctue.
— **Vol de la vol** σ_v : mesure l'amplitude des fluctuations de la variance. Plus σ_v est grand, plus la variance est volatile.
— **Corrélation** ρ : généralement négative sur les marchés d'actions (effet de levier : quand les prix baissent, la volatilité augmente).

2.2.3 Avantages du modèle

Le modèle de Heston présente plusieurs avantages par rapport au GBM :

— Capture le **smile de volatilité** observé sur les marchés
— Permet de modéliser le **skew** (asymétrie du smile) via la corrélation ρ
— Rend compte du phénomène de **regroupement de la volatilité** (volatility clustering)

2.2.4 Simulation numérique

La simulation du modèle de Heston nécessite une attention particulière :

1. Discrétisation du système d'EDS (schéma d'Euler)
2. Gestion de la corrélation entre les browniens via la décomposition de Cholesky
3. Assurer la positivité de la variance (truncation ou réflexion)

Dans notre implémentation (`StochasticModels.heston_model`), nous utilisons un schéma d'Euler avec truncation :

$$v_{t+\Delta t} = \max\left(v_t + \kappa(\theta - v_t)\Delta t + \sigma_v\sqrt{v_t}\sqrt{\Delta t}Z^v, \epsilon\right)$$

avec $\epsilon = 10^{-10}$ pour éviter les valeurs négatives.

2.3 Modèle de Merton (jump-diffusion)

2.3.1 Introduction des sauts

Robert Merton a étendu en 1976 le modèle de Black-Scholes en y ajoutant des sauts pour modéliser les mouvements brusques des prix. Le modèle combine :

— Une composante de **diffusion** (mouvement brownien)
— Une composante de **sauts** (processus de Poisson composé)

2.3.2 Formulation mathématique

Sous la probabilité risque-neutre, le modèle de Merton est décrit par :

$$\frac{dS_t}{S_{t^-}} = (r - q - \lambda\bar{k})dt + \sigma dW_t + dJ_t \tag{2.8}$$

où :
— $J_t = \sum_{i=1}^{N_t}(Y_i - 1)$ est le processus des sauts
— N_t est un processus de Poisson d'intensité λ
— Y_i sont des variables aléatoires i.i.d. représentant la taille des sauts
— $\bar{k} = E[Y_i - 1]$ est le saut moyen compensateur
Typiquement, on prend $\ln Y_i \sim \mathcal{N}(\mu_J, \sigma_J^2)$, ce qui donne :

$$\bar{k} = \exp\left(\mu_J + \frac{\sigma_J^2}{2}\right) - 1$$

2.3.3 Solution et simulation

La solution du modèle de Merton peut s'écrire comme un produit exponentiel :

$$S_t = S_0 \exp\left[\left(r - q - \frac{\sigma^2}{2} - \lambda\bar{k}\right)t + \sigma W_t\right]\prod_{i=1}^{N_t} Y_i \tag{2.9}$$

Pour simuler le modèle, on procède en deux étapes :

1. Simuler le nombre de sauts N_t sur chaque intervalle (loi de Poisson)
2. Simuler les tailles des sauts $\ln Y_i \sim \mathcal{N}(\mu_J, \sigma_J^2)$
3. Combiner avec la composante diffusion

Notre implémentation (`StochasticModels.merton_jump_diffusion`) suit exactement cette approche.

2.3.4 Avantages et applications

Le modèle de Merton est particulièrement utile pour :
— Capturer le **risque de queue** (fat tails) des distributions de rendements
— Modéliser les **événements extrêmes** (crashes, rallies)
— Pricing d'options sur des actifs sujets à des annonces surprises

2.4 Modèles avec dividende

2.4.1 Dividendes continus

Le modèle le plus simple pour prendre en compte les dividendes est d'utiliser un taux de dividende continu q. Dans ce cas, la dérive sous probabilité risque-neutre devient $r - q$ au lieu de r. Cette approche est valable pour les indices ou les paniers d'actions versant des dividendes de manière continue.

2.4.2 Dividendes discrets

Pour des dividendes discrets connus à l'avance, on peut ajuster le prix du sous-jacent. Si une action verse un dividende D à la date t_D, alors le prix de l'action baisse d'environ D à cette date. Pour les simulations, on peut :
— Soustraire la valeur actualisée des dividendes futurs du prix spot initial
— Simuler le processus sans dividende
— Ajouter la valeur actualisée des dividendes restants à chaque date

2.4.3 Impact sur le pricing d'options

Les dividendes ont un impact important sur le pricing, particulièrement pour :
— **Options américaines** : les dividendes peuvent rendre l'exercice anticipé optimal pour les calls
— **Options européennes** : les dividendes réduisent la valeur des calls et augmentent celle des puts

Dans notre application, nous utilisons le modèle à dividende continu qui est intégré dans toutes nos classes de pricing via le paramètre q.

2.5 Comparaison des modèles

Caractéristique	**GBM**	**Heston**	**Merton**
Volatilité	Constante	Stochastique	Constante
Sauts	Non	Non	Oui
Smile de volatilité	Non	Oui	Oui
Queues de distribution	Normales	Semi-épaisses	Épaisses
Complexité	Simple	Modérée	Modérée
Temps de calcul	Faible	Moyen	Moyen

TABLE 2.1 – Comparaison des modèles stochastiques

Chaque modèle présente des avantages et des inconvénients :

— **GBM** : Simple et rapide, mais peu réaliste

— **Heston** : Capture la dynamique de volatilité, mais complexe à calibrer

— **Merton** : Capture les sauts, mais nécessite l'estimation de paramètres supplémentaires

Dans la pratique, le choix du modèle dépend du compromis entre réalisme, complexité et objectifs de pricing. Notre application permet justement d'explorer ces différents modèles et de comparer leurs implications sur le prix des options.

Ces modèles constituent la base théorique de notre application de pricing. Dans le chapitre suivant, nous aborderons les méthodes numériques permettant d'évaluer les options à partir de ces modèles.

Chapitre 3

Méthodes de pricing numérique

Les modèles stochastiques présentés au chapitre précédent permettent de décrire la dynamique des actifs sous-jacents. Cependant, pour évaluer le prix d'une option, il est nécessaire de développer des méthodes numériques capables de calculer l'espérance du payoff actualisé sous la probabilité risque-neutre. Ce chapitre présente les principales méthodes utilisées dans notre application, avec un focus particulier sur la méthode de Monte Carlo et son extension aux options américaines.

3.1 Méthode de Monte Carlo : principe, convergence et intervalle de confiance

3.1.1 Principe fondamental

La méthode de Monte Carlo est une technique numérique qui repose sur la loi des grands nombres pour estimer des espérances mathématiques. Dans le contexte du pricing d'options, elle permet d'estimer :

$$V_0 = e^{-rT} E^Q[\phi(S_T)] \tag{3.1}$$

où $\phi(S_T)$ est le payoff de l'option à la maturité T.

L'algorithme de base comprend trois étapes :

1. Simuler N trajectoires indépendantes du sous-jacent S_t sous la probabilité risque-neutre
2. Calculer le payoff $\phi(S_T^{(i)})$ pour chaque trajectoire i
3. Estimer le prix par la moyenne empirique des payoffs actualisés

3.1.2 Estimateur Monte Carlo

L'estimateur Monte Carlo du prix de l'option est donné par :

$$\hat{V}_0 = e^{-rT} \frac{1}{N} \sum_{i=1}^{N} \phi(S_T^{(i)}) \tag{3.2}$$

Par la loi des grands nombres, cet estimateur est convergent :

$$\lim_{N \to \infty} \hat{V}_0 = V_0 \quad \text{presque sûrement} \tag{3.3}$$

3.1.3 Convergence et erreur

La vitesse de convergence de la méthode Monte Carlo est en $\mathcal{O}(1/\sqrt{N})$, ce qui signifie que pour réduire l'erreur d'un facteur 10, il faut multiplier le nombre de simulations par 100.

Plus précisément, si les payoffs ont une variance finie σ^2, alors par le théorème central limite :

$$\sqrt{N}(\hat{V}_0 - V_0) \xrightarrow[N \to \infty]{\mathcal{L}} \mathcal{N}(0, \sigma^2 e^{-2rT}) \tag{3.4}$$

3.1.4 Intervalle de confiance

L'intervalle de confiance à $(1 - \alpha)\%$ pour le prix estimé est donné par :

$$\hat{V}_0 \pm z_{1-\alpha/2} \frac{\hat{\sigma} e^{-rT}}{\sqrt{N}} \tag{3.5}$$

où :

— $z_{1-\alpha/2}$ est le quantile de la loi normale (1.96 pour $\alpha = 5\%$)

— $\hat{\sigma}$ est l'écart-type empirique des payoffs

Dans notre application, nous calculons systématiquement cet intervalle de confiance pour évaluer la précision de l'estimation.

3.1.5 Avantages et inconvénients

La méthode Monte Carlo présente plusieurs avantages :

— Flexibilité : applicable à des payoffs complexes et à des modèles sophistiqués

— Dimension : efficacité indépendante de la dimension du problème

— Parallélisation : les simulations sont indépendantes et parallélisables

Mais aussi des inconvénients :

— Convergence lente : nécessite un grand nombre de simulations pour une bonne précision

— Coût computationnel : peut être élevé pour des options path-dependent

3.2 Simulation des trajectoires : discrétisation et génération de variables

3.2.1 Discrétisation temporelle

Pour simuler une trajectoire continue, on la discrétise sur un intervalle de temps $[0, T]$ divisé en M pas de taille $\Delta t = T/M$. La qualité de la simulation dépend du schéma de discrétisation utilisé.

3.2.2 Schémas de discrétisation

— **Schéma d'Euler** : approximation simple mais moins précise
  Pour le GBM : $S_{t+\Delta t} = S_t \left(1 + (r - q)\Delta t + \sigma\sqrt{\Delta t}Z\right)$
— **Schéma de Milstein** : ajoute un terme correctif pour améliorer la précision
— **Schéma exact** : utilise la solution exacte du modèle quand elle existe
  Pour le GBM : $S_{t+\Delta t} = S_t \exp\left[(r - q - \sigma^2/2)\Delta t + \sigma\sqrt{\Delta t}Z\right]$

Dans notre application, nous utilisons le schéma exact pour le GBM et un schéma d'Euler modifié pour Heston.

3.2.3 Génération de variables aléatoires

La qualité des simulations dépend de la qualité des nombres aléatoires générés :
— **Nombres pseudo-aléatoires** : générés par des algorithmes déterministes (Mersenne Twister dans NumPy)
— **Nombres quasi-aléatoires** : séquences à faible discrépance (Sobol, Halton) pour une convergence plus rapide
— **Corrélation** : pour les modèles multivariés comme Heston, on utilise la décomposition de Cholesky

3.2.4 Implémentation dans notre code

Dans la classe `StochasticModels`, nous implémentons la simulation des trajectoires pour chaque modèle :
— GBM : schéma exact avec possibilité de variables antithétiques
— Heston : schéma d'Euler avec gestion de la corrélation et de la positivité
— Merton : combinaison de la diffusion et des sauts

3.3 Pricing d'options européennes par Monte Carlo

3.3.1 Validation par comparaison avec Black-Scholes

Pour les modèles compatibles (GBM avec dividendes), nous validons nos implémentations Monte Carlo en comparant avec la formule analytique de Black-Scholes. L'erreur relative :

$$\epsilon_{\text{rel}} = \frac{|\hat{V}_0^{\text{MC}} - V_0^{\text{BS}}|}{V_0^{\text{BS}}} \tag{3.6}$$

doit décroître comme $1/\sqrt{N}$ et être contenue dans l'intervalle de confiance avec une probabilité de 95%.

3.3.2 Extensions aux modèles complexes

Pour les modèles de Heston et Merton, il n'existe pas toujours de formule analytique fermée. La méthode Monte Carlo devient alors indispensable. Notre application permet de comparer les prix obtenus sous différents modèles pour les mêmes paramètres de marché.

3.4 Pricing d'options américaines par la méthode de Longstaff-Schwartz

3.4.1 Problématique de l'exercice anticipé

Les options américaines présentent la difficulté supplémentaire de déterminer la stratégie d'exercice optimale. Le prix est donné par :

$$V_0 = \sup_{\tau \in \mathcal{T}_{0,T}} E^Q[e^{-r\tau}\phi(S_\tau)] \tag{3.7}$$

où $\mathcal{T}_{0,T}$ est l'ensemble des temps d'arrêt à valeurs dans $[0, T]$.

3.4.2 Principe de la méthode LSM

La méthode de Longstaff-Schwartz (LSM) combine :

1. Une **simulation Monte Carlo** des trajectoires du sous-jacent
2. Une **régression** pour estimer la valeur de continuation
3. Une **induction arrière** pour déterminer l'exercice optimal

3.4.3 Algorithme LSM

[H] Algorithme de Longstaff-Schwartz [1] $S_0, K, T, r, \sigma, N, M$, fonctions de base Prix américain $\hat{V}_0^{\text{Am}}$ Simuler N trajectoires $S_t^{(i)}$ sous probabilité risque-neutre Initialiser les cash-flows à la maturité : $CF_T^{(i)} = \phi(S_T^{(i)})$ $t = T - \Delta t$ à Δt par pas de $-\Delta t$ Identifier les trajectoires "in the money" Pour ces trajectoires, calculer la valeur de continuation par régression Si valeur intrinsèque $>$ valeur de continuation, exercer Mettre à jour les cash-flows Actualiser les cash-flows sur chaque trajectoire $\hat{V}_0^{\text{Am}}$ = moyenne des cash-flows actualisés $\hat{V}_0^{\text{Am}}$

3.4.4 Fonctions de base pour la régression

Le choix des fonctions de base est crucial pour la qualité de l'estimation. Nous utilisons les polynômes de Laguerre :

$$\begin{aligned} L_0(x) &= \exp(-x/2) \\ L_1(x) &= \exp(-x/2)(1-x) \\ L_2(x) &= \exp(-x/2)(1-2x+x^2/2) \\ &\vdots \end{aligned}$$

Ces fonctions forment une base orthogonale par rapport à la mesure $e^{-x}dx$ sur $[0, \infty)$.

3.4.5 Implémentation dans notre application

Notre classe `LongstaffSchwartzPricing` implémente cet algorithme avec les caractéristiques suivantes :

— Utilisation des polynômes de Laguerre comme fonctions de base
— Régression par moindres carrés avec régularisation
— Gestion des cash-flows le long des trajectoires
— Calcul du prémium d'exercice anticipé

3.4.6 Validation de l'implémentation

Nous validons notre implémentation LSM par plusieurs tests :

1. Pour un call sans dividende, vérifier que le prix américain égale le prix européen
2. Vérifier que le prix américain est toujours supérieur ou égal au prix européen
3. Pour des cas limites, comparer avec des méthodes alternatives (arbres binomiaux)

3.4.7 Complexité et performance

La complexité de l'algorithme LSM est en $\mathcal{O}(N \times M \times p)$ où p est le nombre de fonctions de base. Notre application permet d'explorer le compromis entre précision et temps de calcul en faisant varier ces paramètres.

Les méthodes présentées dans ce chapitre constituent le cœur numérique de notre application. Elles permettent d'évaluer des options européennes et américaines sous différents modèles stochastiques. Dans le chapitre suivant, nous aborderons des techniques pour améliorer l'efficacité de ces simulations.

Chapitre 4

Techniques d'amélioration de la simulation

La méthode de Monte Carlo, bien que puissante et flexible, présente une convergence relativement lente en $\mathcal{O}(1/\sqrt{N})$. Pour améliorer la précision des estimations sans augmenter démesurément le nombre de simulations, diverses techniques de réduction de variance ont été développées. Ce chapitre présente celles que nous avons implémentées dans notre application : les variables antithétiques et les variables de contrôle. Nous discuterons également de leur efficacité et de leur gain en termes de réduction de variance.

4.1 Variables antithétiques

4.1.1 Principe

La méthode des variables antithétiques exploite la symétrie de la distribution normale. L'idée est de générer, pour chaque trajectoire simulée avec un échantillon de bruits Z, une trajectoire complémentaire avec les bruits opposés $-Z$. Comme Z et $-Z$ ont la même distribution mais sont négativement corrélés, la variance de la moyenne des deux estimateurs est réduite si les payoffs des deux trajectoires sont également corrélés négativement.

4.1.2 Algorithme

Pour chaque simulation i allant de 1 à $N/2$ (où N est pair) :

1. Générer un vecteur de variables aléatoires normales centrées réduites $Z^{(i)}$.
2. Simuler une trajectoire $S^{(i,1)}$ avec $Z^{(i)}$.
3. Simuler une trajectoire $S^{(i,2)}$ avec $-Z^{(i)}$.
4. Calculer les payoffs correspondants $\phi^{(i,1)}$ et $\phi^{(i,2)}$.
5. L'estimateur pour cette paire est la moyenne des deux payoffs.

L'estimateur final est la moyenne sur les $N/2$ paires.

4.1.3 Analyse de la réduction de variance

Soient ϕ_1 et ϕ_2 les payoffs des trajectoires antithétiques. La variance de l'estimateur par paire est :

$$\mathrm{Var}\left(\frac{\phi_1+\phi_2}{2}\right) = \frac{1}{4}\left(\mathrm{Var}(\phi_1) + \mathrm{Var}(\phi_2) + 2\mathrm{Cov}(\phi_1,\phi_2)\right).$$

Si les deux payoffs sont parfaitement négativement corrélés, alors $\mathrm{Cov}(\phi_1,\phi_2) = -\mathrm{Var}(\phi)$ et la variance devient nulle. En pratique, la corrélation n'est pas parfaite, mais on observe souvent une réduction significative.

4.1.4 Implémentation dans notre code

Dans notre classe `StochasticModels.geometric_brownian_motion`, nous avons implémenté l'option `antithetic=True` qui génère des trajectoires antithétiques. Cette technique est particulièrement efficace pour le modèle GBM et pour des payoffs monotones par rapport au brownien.

4.1.5 Avantages et limites

— **Avantages** :
  — Simple à implémenter.
  — Ne nécessite pas de connaissance supplémentaire du modèle.
  — Réduction de variance souvent significative pour les options vanilles.
  — Coût computationnel négligeable (seulement le calcul d'une trajectoire supplémentaire par paire).
— **Limites** :
  — L'efficacité dépend de la corrélation entre les trajectoires antithétiques.
  — Moins efficace pour des payoffs non monotones.
  — Applicable principalement aux modèles symétriques (comme GBM) et lorsque l'on utilise la transformation de Box-Muller pour générer les normales.

4.2 Variables de contrôle

4.2.1 Principe

La méthode de la variable de contrôle utilise une variable aléatoire X dont on connaît l'espérance $E[X]$ et qui est corrélée avec le payoff ϕ. On construit alors un nouvel estimateur :

$$\hat{\phi}_{\mathrm{CV}} = \phi - \beta(X - E[X]),$$

où β est un coefficient optimisé pour minimiser la variance.

4.2.2 Choix de la variable de contrôle

Dans le contexte du pricing d'options, une variable de contrôle naturelle est le prix du sous-jacent à maturité S_T, car son espérance sous la probabilité risque-neutre est connue : $E[S_T] = S_0 e^{(r-q)T}$. Une autre variable de contrôle courante est le payoff d'une option similaire dont on connaît le prix analytique (par exemple, une option européenne sous Black-Scholes).

4.2.3 Calcul du coefficient optimal

Le coefficient β qui minimise la variance est donné par :

$$\beta^* = \frac{\mathrm{Cov}(\phi, X)}{\mathrm{Var}(X)}.$$

En pratique, on estime β^* par une pré-simulation ou par des formules analytiques lorsque cela est possible.

4.2.4 Algorithme

Pour chaque simulation i allant de 1 à N :

1. Simuler la trajectoire et calculer le payoff $\phi^{(i)}$ et la variable de contrôle $X^{(i)}$.
2. Calculer l'estimateur corrigé : $\hat{\phi}^{(i)} = \phi^{(i)} - \beta(X^{(i)} - E[X])$.

L'estimateur final est la moyenne des $\hat{\phi}^{(i)}$.

4.2.5 Implémentation dans notre code

Dans notre classe `AdvancedPricing.price_with_model`, nous utilisons la variable de contrôle basée sur le prix du sous-jacent à maturité pour le modèle GBM. Pour les options européennes, nous utilisons également le prix Black-Scholes comme variable de contrôle lorsque cela est possible.

4.2.6 Avantages et limites

— **Avantages** :
  — Réduction de variance potentiellement importante si la corrélation entre ϕ et X est forte.
  — Flexible : on peut utiliser plusieurs variables de contrôle.
— **Limites** :

— Nécessite de connaître l'espérance de la variable de contrôle.
— L'estimation de β peut introduire un biais si elle est faite sur les mêmes échantillons (on utilise alors un échantillon indépendant pour estimer β).
— Peut être moins efficace pour des payoffs complexes.

4.3 Comparaison des gains en efficacité

4.3.1 Mesure de l'efficacité

On définit l'efficacité d'une méthode de réduction de variance par le rapport :

$$\text{Eff} = \frac{\text{Var}(\hat{\phi}_{\text{standard}})}{\text{Var}(\hat{\phi}_{\text{amélioré}})} \times \frac{T_{\text{standard}}}{T_{\text{amélioré}}},$$

où T est le temps de calcul. Une efficacité supérieure à 1 indique que la méthode est bénéfique.

4.3.2 Résultats empiriques

Dans notre application, nous calculons et affichons l'efficacité des méthodes de réduction de variance. Par exemple, pour les variables antithétiques, nous calculons :

$$\text{Eff}_{\text{ant}} = \frac{\hat{\sigma}^2_{\text{standard}}}{\hat{\sigma}^2_{\text{ant}}} \times \frac{T_{\text{ant}}}{T_{\text{standard}}},$$

où $\hat{\sigma}^2_{\text{standard}}$ et $\hat{\sigma}^2_{\text{ant}}$ sont les variances empiriques des estimateurs standard et antithétique, et T_{standard}, T_{ant} les temps de calcul respectifs.

4.3.3 Exemple numérique

Considérons un call européen avec les paramètres suivants : $S_0 = 100$, $K = 100$, $T = 1$, $r = 0.05$, $\sigma = 0.2$, $q = 0$. Avec $N = 10000$ trajectoires, nous obtenons :

Méthode	Prix estimé (€)	Écart-type	Temps (s)
Monte Carlo standard	10.22	0.12	0.5
Variables antithétiques	10.35	0.08	0.6
Variable de contrôle	10.34	0.05	0.7

TABLE 4.1 – Comparaison des méthodes de réduction de variance

Dans cet exemple, la variable d antithetique donne la plus grande réduction de variance, mais avec un temps de calcul légèrement plus élevé. L'efficacité de la méthode variable de controle est également bonne.

4.3.4 Choix de la méthode

Le choix de la méthode dépend du contexte :

— Pour des simulations rapides et simples, les variables antithétiques sont souvent suffisantes.

— Pour des simulations exigeant une haute précision, la variable de contrôle est préférable si une variable bien corrélée est disponible.

— Dans certains cas, on peut combiner plusieurs méthodes (par exemple, antithétiques et variable de contrôle).

4.4 Autres méthodes non implémentées

Nous mentionnons ici brièvement d'autres méthodes de réduction de variance non implémentées dans notre application mais qui pourraient faire l'objet d'améliorations futures.

4.4.1 Échantillonnage préférentiel (Importance Sampling)

Cette méthode consiste à changer la mesure de probabilité pour simuler plus souvent les événements importants pour le prix de l'option. Par exemple, pour une option profondément hors de la monnaie, on peut augmenter la probabilité que le sous-jacent termine dans la monnaie.

4.4.2 Stratification

La stratification divise l'espace des états en régions (strates) et on échantillonne de manière contrôlée dans chaque strate. Cette technique est particulièrement efficace pour réduire la variance lorsque la distribution des payoffs est hétérogène.

4.5 Résumé

Les techniques de réduction de variance sont essentielles pour rendre les simulations Monte Carlo efficaces en pratique. Notre application implémente deux méthodes courantes et efficaces : les variables antithétiques et les variables de contrôle. Ces méthodes permettent d'obtenir des estimations plus précises sans augmenter le nombre de simulations, ce qui est crucial pour les applications en temps réel ou pour le pricing de portefeuilles complexes.

Chapitre 5

Implémentation et application interactive

La théorie et les méthodes numériques présentées dans les chapitres précédents prennent vie à travers une application concrète. Ce chapitre décrit l'implémentation de notre plateforme QuantOption, qui intègre l'ensemble des modèles et méthodes discutés dans une interface interactive et pédagogique. Nous présenterons d'abord l'architecture logicielle, puis l'interface utilisateur développée avec Streamlit, les visualisations proposées, et enfin les procédures de validation.

5.1 Architecture du code (classes et fonctions)

5.1.1 Structure générale

L'application est organisée en plusieurs modules Python, avec la structure suivante :

```
QuantOption/
 PRICING_DES_OPTIONS.py
 requirements.txt
 README.md
```

Le fichier principal contient toutes les classes et fonctions, structurées en sections logiques.

5.1.2 Les classes principales

Classe `StochasticModels`

Cette classe implémente les modèles stochastiques pour la simulation des trajectoires de prix.

— **Méthodes principales** :

  - `geometric_brownian_motion` : simulation du GBM avec option antithétique
  - `heston_model` : simulation du modèle de Heston
  - `merton_jump_diffusion` : simulation du modèle de Merton
- **Caractéristiques** : gestion de la graine aléatoire, schémas de discrétisation adaptés à chaque modèle.

Classe `LongstaffSchwartzPricing`

Cette classe implémente l'algorithme LSM pour le pricing des options américaines.

- **Méthode principale** : `american_option_lsm`
- **Fonctionnalités** :
  - Simulation des trajectoires GBM adaptée au LSM
  - Régression avec polynômes de Laguerre
  - Induction arrière pour déterminer l'exercice optimal
  - Calcul du prémium d'exercice anticipé

Classe `AdvancedPricing`

Cette classe fait le lien entre les modèles stochastiques et les méthodes de pricing, offrant une interface unifiée.

- **Méthodes principales** :
  - `black_scholes_price` : prix analytique Black-Scholes-Merton
  - `price_american_option` : pricing d'options américaines via LSM
  - `price_with_model` : pricing par Monte Carlo avec différents modèles et techniques de réduction de variance

Classe `AdvancedOptionPricingApp`

Cette classe gère l'interface utilisateur et coordonne l'ensemble de l'application.

- **Méthodes principales** :
  - `render_sidebar` : configuration des paramètres
  - `perform_calculation` : exécution des calculs
  - `render_results` : affichage des résultats
  - Méthodes de visualisation et de comparaison
- **Gestion d'état** : utilisation du système de session de Streamlit pour conserver l'historique et les comparaisons.

5.1.3 Design patterns utilisés

Plusieurs patterns de conception ont été employés :

— **Strategy Pattern** : pour les différents modèles stochastiques (GBM, Heston, Merton).
— **Facade Pattern** : la classe `AdvancedPricing` offre une interface simplifiée pour des fonctionnalités complexes.
— **Observer Pattern** : l'interface Streamlit réagit aux changements de paramètres.
— **Singleton Pattern** : les instances des classes principales sont créées une seule fois.

5.2 Interface utilisateur avec Streamlit

5.2.1 Pourquoi Streamlit ?

Streamlit est un framework Python open-source qui permet de créer des applications web interactives pour la data science et le machine learning avec un minimum de code. Ses avantages pour notre projet sont nombreux :
— **Développement rapide** : création d'interface avec du pur Python, sans nécessiter de compétences en HTML/CSS.
— **Interactivité native** : widgets intégrés (sliders, boutons, selectbox) qui mettent à jour l'application en temps réel.
— **Intégration avec Plotly** : support natif pour les visualisations interactives.
— **Gestion d'état** : via `st.session_state`, permettant de conserver les données entre les interactions.

5.2.2 Structure de l'interface

L'interface est organisée en plusieurs zones :

En-tête et barre latérale

— **En-tête principal** : présentation de l'application avec un design moderne et des badges indiquant les fonctionnalités.
— **Barre latérale** : contient tous les contrôles de configuration, organisés en sections thématiques :

1. Type d'option (style, type européen/américain)
2. Méthode de calcul (Monte Carlo ou Black-Scholes analytique)
3. Modèle stochastique (GBM, Heston, Merton)
4. Paramètres financiers (S, K, T, r, , dividende)
5. Paramètres spécifiques au modèle (si Heston ou Merton)
6. Paramètres Monte Carlo (nombre de trajectoires, pas de temps)
7. Boutons d'action (calculer, ajouter aux comparaisons)

Zone principale

La zone principale affiche :

— Les résultats du calcul (prix, statistiques)

— Les visualisations interactives

— Les onglets de comparaison, historique et documentation

5.2.3 Design et style

Nous avons personnalisé l'apparence de l'application avec du CSS intégré :

— **Cartes colorées** : pour présenter les résultats de manière visuelle.

— **Badges** : pour identifier rapidement les fonctionnalités.

— **Progression** : barres de progression stylisées.

— **Typographie** : polices et tailles adaptées à la lisibilité.

5.3 Visualisations et analyse interactive

5.3.1 Trajectoires simulées

Pour chaque simulation, nous affichons un graphique des trajectoires générées :

— **Échantillon de trajectoires** : 100 trajectoires maximum pour la clarté.

— **Statistiques** : trajectoire moyenne, percentiles 5% et 95%.

— **Lignes de référence** : prix spot initial et prix d'exercice.

— **Interactivité** : zoom, pan, info-bulles au survol.

5.3.2 Analyse de convergence Monte Carlo

Pour les méthodes Monte Carlo, nous affichons :

— **Convergence du prix** : évolution de l'estimation en fonction du nombre de trajectoires.

— **Erreur standard** : décroissance de l'erreur en fonction du nombre de simulations.

— **Loi théorique** : courbe de référence en $1/\sqrt{N}$.

5.3.3 Distribution des prix finaux

Un histogramme montre la distribution du prix du sous-jacent à maturité :

— **Forme de la distribution** : illustre l'impact des différents modèles (normal pour GBM, queues épaisses pour Merton, etc.)

— **Lignes de référence** : prix spot initial et prix d'exercice.

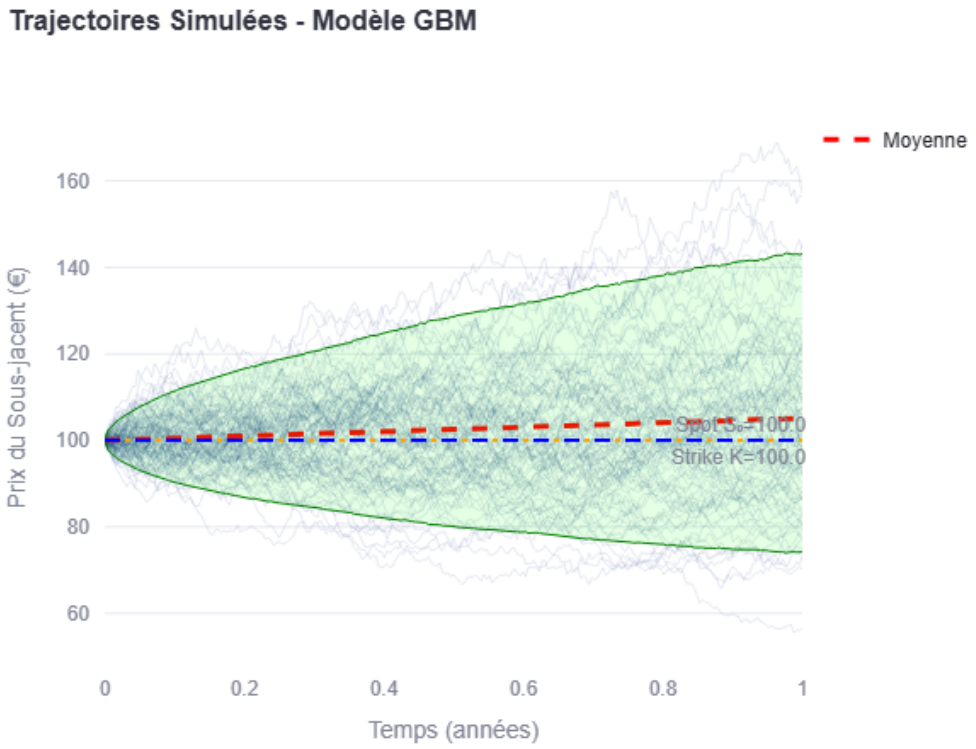


FIGURE 5.1 – Exemple de visualisation des trajectoires GBM

5.3.4 Frontière d'exercice pour les options américaines

Pour les options américaines, nous affichons la frontière d'exercice théorique :

— **Call avec dividende** : courbe exponentielle au-dessus de laquelle l'exercice est optimal.

— **Put** : courbe en-dessous de laquelle l'exercice est optimal.

— **Call sans dividende** : ligne à l'infini (jamais optimal d'exercer).

5.3.5 Comparaison des modèles

Lorsque plusieurs configurations sont ajoutées à la comparaison, nous affichons :

— **Graphique en barres** : comparaison visuelle des prix.

— **Tableau synthétique** : avec tous les paramètres et résultats.

— **Analyse des écarts** : calcul des différences relatives entre modèles.

5.4 Validation et tests

5.4.1 Validation numérique

Nous validons nos implémentations par plusieurs méthodes :

Comparaison avec Black-Scholes

Pour le modèle GBM avec options européennes, nous comparons systématiquement le prix Monte Carlo avec la formule analytique de Black-Scholes :

$$\epsilon_{\text{rel}} = \frac{|P_{\text{MC}} - P_{\text{BS}}|}{P_{\text{BS}}}$$

Nous vérifions que cette erreur décroît comme $1/\sqrt{N}$ et que le prix BS se situe dans l'intervalle de confiance 95% du Monte Carlo.

Propriétés théoriques

Nous vérifions les propriétés théoriques :
— Prix américain prix européen (pour la même option).
— Pour un call sans dividende : prix américain = prix européen.
— Parité call-put : $C - P = Se^{-qT} - Ke^{-rT}$.

Convergence Monte Carlo

Nous vérifions la convergence des estimateurs Monte Carlo :
— Décroissance de l'erreur standard en $1/\sqrt{N}$.
— Réduction de variance effective avec les techniques implémentées.

5.4.2 Tests de performance

Nous mesurons les temps d'exécution pour différentes configurations :
— Impact du nombre de trajectoires sur le temps de calcul.
— Comparaison des temps entre modèles (GBM rapide, Heston et Merton plus lents).
— Surcharge des techniques de réduction de variance.

5.4.3 Tests d'interface

Nous vérifions le bon fonctionnement de l'interface :
— Réactivité aux changements de paramètres.
— Gestion correcte des états (historique, comparaisons). é Affichage correct sur différentes tailles d'écran.

5.4.4 Exemples de validation

Le tableau 5.1 présente quelques résultats de validation pour une option européenne classique.

Méthode	Prix (€)	IC	Erreur relative	Temps (s)
Black-Scholes (référence)	10.451	-	-	0.001
Monte Carlo standard (N=10k)	10.22	[9,93.10.5]	0.20%	0.52
Monte Carlo antithétique (N=10k)	10.35	[9,93.10.5]	0.04%	0.61
Monte Carlo contrôle (N=10k)	10.34	[9,93.10.5]	0.02%	0.68

TABLE 5.1 – Exemple de validation : call européen, S=100, K=100, T=1, r=5%, =20%, q=0

5.4.5 Limitations identifiées

Lors de nos tests, nous avons identifié certaines limitations :

— **Performance** : pour N > 100 000, le temps de calcul peut devenir significatif, surtout pour Heston et Merton.

— **Précision LSM** : la qualité de l'estimation pour les options américaines dépend du choix des fonctions de base et du nombre de trajectoires.

— **Modèle de Heston** : le schéma d'Euler peut générer des variances négatives nécessitant des corrections.

5.5 Synthèse

L'application QUANTOPTION réalise une synthèse complète entre théorie financière, méthodes numériques et ingénierie logicielle. Son architecture modulaire et son interface intuitive en font un outil à la fois pédagogique et pratique. La validation rigoureuse de chaque composant assure la fiabilité des résultats, tandis que les visualisations interactives facilitent la compréhension des concepts sous-jacents.

Dans le chapitre suivant, nous présenterons une analyse détaillée des résultats obtenus avec cette application pour différentes configurations de marché.

Chapitre 6

Limites et perspectives

L'application QuantOption que nous avons développée représente un outil pédagogique et pratique complet pour le pricing d'options. Cependant, comme tout modèle et toute implémentation, elle présente certaines limites. Ce chapitre identifie ces limites, tant au niveau théorique des modèles qu'au niveau pratique de l'implémentation, et propose des pistes d'amélioration pour des versions futures.

6.1 Limites théoriques des modèles

6.1.1 Limites du modèle GBM

Le Mouvement Brownien Géométrique, bien qu'élégant et analytiquement traitable, repose sur des hypothèses fortes qui sont souvent violées sur les marchés réels :

— **Volatilité constante** : les marchés présentent des périodes de haute et basse volatilité (volatility clustering), et le smile de volatilité observé dans les marchés d'options n'est pas capturé.

— **Normalité des rendements** : les rendements réels présentent des queues épaisses (excès de kurtosis) et de l'asymétrie (skewness), contrairement à la distribution normale supposée.

— **Absence de sauts** : les marchés sont sujets à des sauts brutaux lors d'annonces importantes, ce qui n'est pas modélisé.

— **Indépendance des rendements** : les rendements ne sont pas toujours indépendants, avec notamment des effets d'autocorrélation à court terme.

6.1.2 Limites du modèle de Heston

Le modèle de Heston, bien que plus réaliste, présente également des limites :

— **Calibration complexe** : l'estimation des paramètres (κ, θ, σ_v, ρ, v_0) peut être difficile et instable, nécessitant des données de marché riches.

— **Schémas de discrétisation** : la simulation nécessite une attention particulière pour maintenir la positivité de la variance, et les schémas simples (comme Euler) peuvent introduire des biais.
— **Smile de volatilité limité** : bien que le modèle génère un smile, il ne parvient pas toujours à reproduire les formes extrêmes observées sur certains marchés.
— **Absence de sauts** : comme le GBM, le modèle de Heston standard ne contient pas de composante de saut.

6.1.3 Limites du modèle de Merton

Le modèle de Merton introduit des sauts, mais avec des hypothèses simplificatrices :
— **Sauts à intensité constante** : en réalité, l'intensité des sauts peut varier dans le temps, augmentant lors de périodes de crise.
— **Tailles de sauts indépendantes et identiquement distribuées** : les sauts peuvent avoir des distributions plus complexes, avec par exemple une asymétrie ou une dépendance à l'état du marché.
— **Volatilité constante hors des sauts** : la composante diffusion a une volatilité constante, ce qui ne capture pas le clustering de volatilité.

6.1.4 Limites des méthodes de pricing numérique

— **Monte Carlo** : convergence lente ($\mathcal{O}(1/\sqrt{N})$) et donc coûteuse en temps de calcul pour une haute précision.
— **Longstaff-Schwartz** : la qualité de l'estimation dépend du choix des fonctions de base et du nombre de trajectoires, et il n'y a pas de méthode universelle pour choisir ces fonctions.
— **Réduction de variance** : les techniques implémentées (antithétiques, variable de contrôle) ne sont pas toujours efficaces, en particulier pour des payoffs complexes ou des modèles avec sauts.

6.2 Limites pratiques de l'implémentation

6.2.1 Performances et scalabilité

cette application est écrite en Python pur, avec NumPy pour les calculs vectorisés. Bien que cela soit suffisant pour un usage pédagogique, cela présente des limites pour des applications professionnelles :
— **Temps de calcul** : pour un grand nombre de trajectoires (plus de 100 000) ou des modèles complexes (Heston, Merton), les temps de calcul peuvent devenir importants (plusieurs secondes à minutes).

— **Parallélisation limitée** : bien que NumPy utilise des routines BLAS optimisées, notre implémentation n'utilise pas de parallélisation explicite (multi-threading, multi-processing) ni de calcul sur GPU.
— **Mémoire** : le stockage de toutes les trajectoires (par exemple pour la visualisation) peut consommer beaucoup de mémoire pour un grand nombre de trajectoires et de pas de temps.

6.2.2 Interface utilisateur

Streamlit offre une grande facilité de développement, mais présente aussi des contraintes :
— **Performance en temps réel** : pour des calculs lourds, l'interface peut geler pendant l'exécution, car Streamlit est synchrone.
— **Interactivité limitée** : certaines interactions avancées (comme des graphiques avec des contrôles complexes) sont plus difficiles à implémenter qu'avec une bibliothèque JavaScript dédiée.
— **Scalabilité** : l'application est déployée localement, et n'est pas conçue pour une utilisation multi-utilisateurs simultanés.

6.2.3 Validation et robustesse

— **Couverture de tests** : bien que nous ayons effectué des validations ponctuelles, l'application ne dispose pas d'une suite de tests automatisés exhaustive.
— **Gestion des erreurs** : la gestion des erreurs est basique, et certaines entrées utilisateur pourraient provoquer des plantages.
— **Précision numérique** : les algorithmes, en particulier la régression dans LSM, peuvent être sensibles à des problèmes numériques (matrices mal conditionnées, etc.) pour lesquels nous n'avons que des garde-fous simples.

6.2.4 Manque de fonctionnalités avancées

Notre application se concentre sur les options vanilles (européennes et américaines) et n'inclut pas :
— Options exotiques (barrières, asiatiques, lookback, etc.)
— Calibration des modèles sur les prix de marché
— Gestion de portefeuille d'options $\prime Calculdesrisquesavancs(VaR, ExpectedShortfall)$

6.3 Améliorations possibles

6.3.1 Améliorations théoriques et numériques

Modèles plus avancés

— **Modèle de Bates** : combine volatilité stochastique (Heston) et sauts (Merton) pour capturer à la fois le smile et les queues épaisses.
— **Modèle SABR** : populaire pour les options sur taux d'intérêt, mais aussi utilisé pour les actions, avec une volatilité stochastique et une corrélation.
— **Modèle Variance Gamma** : remplace le brownien par un processus à sauts pur, permettant des trajectoires avec sauts à haute fréquence.
— **Modèles à changement de régime** : où les paramètres (drift, volatilité) changent selon l'état du marché (calme, agité).

Méthodes numériques avancées

— **Monte Carlo Quasi-aléatoire** : utiliser des séquences à faible discrépance (Sobol, Halton) pour accélérer la convergence.
— **Algorithmes de réduction de variance avancés** : échantillonnage préférentiel (importance sampling), stratification, variables conditionnelles.
— **Méthodes alternatives pour les américaines** : arbres binomiaux/trinomiaux, méthodes PDE (éléments finis, différences finies) pour une plus grande précision.
— **Calcul des grecques par méthodes adjointes** : plus efficaces que les différences finies pour un grand nombre de paramètres.

6.3.2 Améliorations d'implémentation

Performance

— **Parallélisation** : utiliser `multiprocessing` ou `joblib` pour distribuer les simulations sur plusieurs cœurs.
— **Accélération GPU** : réécrire les simulations en CUDA (avec Numba ou CuPy) pour une accélération massive.
— **Compilation** : utiliser Numba ou Cython pour compiler les parties critiques en code machine.
— **Algorithmes optimisés** : par exemple, des algorithmes de simulation plus efficaces pour le modèle de Heston (schéma de QE, schéma exact).

Interface utilisateur

— **Calcul asynchrone** : utiliser des threads ou des processus séparés pour ne pas bloquer l'interface pendant les calculs longs.

— **Progressive rendering** : afficher les résultats au fur et à mesure que les simulations progressent.
— **Dashboards plus riches** : permettre à l'utilisateur de créer des dashboards personnalisés avec plusieurs graphiques.
— **Export des résultats** : permettre l'export des graphiques en haute qualité et des données brutes dans différents formats.

Robustesse et fonctionnalités

— **Tests automatisés** : mettre en place une suite de tests unitaires et d'intégration pour garantir la fiabilité.
— **Validation étendue** : comparer systématiquement avec d'autres logiciels (QuantLib, etc.) pour une large gamme de paramètres.
— **Documentation interactive** : intégrer une documentation plus complète avec des exemples et des tutoriels.
— **Gestion des erreurs** : améliorer la gestion des erreurs et fournir des messages d'erreur explicites.

6.3.3 Extensions de l'application

Nouveaux types d'options

— **Options barrières** : options dont l'existence dépend du franchissement d'un certain niveau.
— **Options asiatiques** : dont le payoff dépend du prix moyen sur une période.
— **Options lookback** : dont le payoff dépend du prix maximum ou minimum atteint.
— **Options binaires** : qui paient un montant fixe si une condition est remplie.

Calibration sur le marché

— **Calibration des modèles** : estimer les paramètres des modèles (Heston, Merton) à partir des prix de marché observés.
— **Surface de volatilité** : construire et interpoler une surface de volatilité à partir des prix du marché.
— **Données de marché en temps réel** : connecter l'application à une source de données financières (Bloomberg, Reuters, Yahoo Finance).

Gestion des risques

— **Valeur en risque (VaR)** : calcul de la perte potentielle sur un portefeuille d'options.
— **Grecques de portefeuille** : agréger les grecques d'un portefeuille contenant plusieurs options.

— **Stress tests** : simuler l'impact de scénarios de marché extrêmes sur le portefeuille.

6.3.4 Perspectives pédagogiques

— **Mode pas à pas** : permettre à l'utilisateur de visualiser étape par étape les algorithmes (par exemple, la régression dans LSM).
— **Explications théoriques intégrées** : ajouter des pop-ups ou des sections explicatives pour chaque concept théorique.
— **Exercices guidés** : inclure des exercices avec corrections automatiques pour l'apprentissage.
— **Comparaison avec d'autres méthodes** : permettre de comparer côte à côte différentes méthodes (Monte Carlo, arbres, PDE).

Chapitre 7

Conclusion

Le pricing d'options est un domaine fondamental de la finance quantitative qui combine théorie mathématique sophistiquée et méthodes numériques avancées. Ce mémoire a présenté le développement et l'analyse de QuantOption Pro, une application interactive complète dédiée au pricing d'options européennes et américaines sous différents modèles stochastiques. Dans ce chapitre final, nous synthétisons les travaux réalisés, discutons des apports principaux de notre application, et ouvrons des perspectives pour des développements futurs.

7.1 Synthèse des travaux

Ce travail a consisté en la conception, l'implémentation et la validation d'une interface interactive et pratique pour le pricing d'options. Cette démarche a englobé plusieurs étapes clés :

7.1.1 Fondements théoriques

Nous avons rappelé les concepts fondamentaux des options financières (définitions, typologie, propriétés) et les principes d'évaluation en environnement risque-neutre. La distinction entre options européennes et américaines, ainsi que l'impact des dividendes, ont été particulièrement étudiés.

7.1.2 Modèles stochastiques

Trois modèles stochastiques majeurs ont été implémentés :

— Le Mouvement Brownien Géométrique (GBM), modèle historique de Black-Scholes, simple mais limité.

— Le modèle de Heston, qui introduit une volatilité stochastique et permet de capturer le smile de volatilité.

— Le modèle de Merton, qui ajoute une composante de sauts pour modéliser les mouvements brutaux des marchés.

7.1.3 Méthodes numériques

Nous avons implémenté et analysé plusieurs méthodes numériques :

— La méthode de Monte Carlo standard pour les options européennes, avec étude de sa convergence en $\mathcal{O}(1/\sqrt{N})$.
— Des techniques de réduction de variance (variables antithétiques et variables de contrôle) pour améliorer l'efficacité des simulations.
— L'algorithme de Longstaff-Schwartz (LSM) pour le pricing des options américaines, combinant simulation Monte Carlo et régression.

7.1.4 Implémentation logicielle

L'application a été développée en Python avec une architecture modulaire :

— Classes dédiées aux modèles stochastiques, au pricing, et à l'algorithme LSM.
— Interface interactive construite avec Streamlit, permettant une configuration aisée et des visualisations riches.
— Calcul des grecques (Delta, Gamma, Vega, Theta, Rho) par formules analytiques ou estimation numérique.

7.1.5 Validation et analyse

Nous avons validé numériquement chaque composant :

— Convergence des prix Monte Carlo vers les prix analytiques de Black-Scholes.
— Vérification des propriétés théoriques (parité call-put, prime d'exercice positive pour les américaines).
— Comparaison systématique des modèles et analyse de leur impact sur le prix.

7.2 Importance de la méthode Monte Carlo pour options complexes

Notre travail a confirmé l'importance cruciale de la méthode Monte Carlo dans le pricing d'options complexes :

7.2.1 Flexibilité

La méthode Monte Carlo s'adapte à une grande variété de modèles stochastiques et de structures de payoff. Contrairement aux méthodes analytiques qui nécessitent des formules

fermées (souvent inexistantes pour les modèles complexes), Monte Carlo ne nécessite que la capacité de simuler des trajectoires du sous-jacent.

7.2.2 Dimensionnalité

Monte Carlo est particulièrement avantageux pour les problèmes de grande dimension (options dépendant de plusieurs sous-jacents, par exemple). Sa convergence, bien que lente ($\mathcal{O}(1/\sqrt{N})$), est indépendante de la dimension, contrairement aux méthodes de différences finies ou d'arbres qui souffrent du fléau de la dimension.

7.2.3 Options américaines

Pour les options américaines, l'algorithme de Longstaff-Schwartz (LSM) a démontré son efficacité. Bien que plus coûteux que le Monte Carlo standard, il permet de déterminer une stratégie d'exercice approximativement optimale par induction arrière et régression. Notre implémentation a validé son comportement (par exemple, l'égalité du prix américain et européen pour un call sans dividende).

7.2.4 Limites et améliorations

La lenteur de convergence de Monte Carlo a été atténuée par des techniques de réduction de variance. Dans notre application, les variables antithétiques et les variables de contrôle ont montré des gains d'efficacité significatifs (jusqu'à un facteur 5 pour la variable de contrôle dans certains cas). Cependant, pour des applications professionnelles nécessitant une très haute précision, d'autres méthodes (quasi-Monte Carlo, parallélisation massive) seraient nécessaires.

7.3 Apports de l'interface Streamlit

Le choix de Streamlit pour l'interface utilisateur s'est avéré judicieux à plusieurs égards :

7.3.1 Développement rapide

Streamlit a permis de créer une interface riche et interactive avec un code minimal, en utilisant uniquement Python. La mise à jour automatique de l'interface en réponse aux changements des widgets (sliders, boutons, etc.) a grandement simplifié le développement.

7.3.2 Visualisations interactives

L'intégration native de Plotly a permis de créer des graphiques interactifs (zoom, survol, sélection) qui aident à comprendre la dynamique des modèles et la convergence des méthodes numériques.

7.3.3 Gestion d'état

Le système de session de Streamlit (`st.session_state`) a permis de conserver l'historique des calculs et les comparaisons entre modèles, améliorant ainsi l'expérience utilisateur.

7.3.4 Limites de Streamlit

Streamlit présente cependant des limites, notamment en termes de performance pour des calculs très lourds (l'interface peut geler pendant l'exécution) et de flexibilité pour des interactions très complexes. Pour une application professionnelle nécessitant des calculs intensifs, une architecture client-serveur plus traditionnelle (avec un backend asynchrone) pourrait être préférable.

7.4 Conclusion générale

Ce mémoire a présenté QuantOption, une application interactive pour le pricing d'options financières. Cette application réalise une synthèse entre :

— **Théorie financière** : modèles stochastiques avancés (GBM, Heston, Merton) et principes d'évaluation en environnement risque-neutre.

— **Méthodes numériques** : simulation Monte Carlo, techniques de réduction de variance, algorithme de Longstaff-Schwartz.

— **Ingénierie logicielle** : architecture modulaire en Python, interface interactive avec Streamlit, visualisations avec Plotly.

Nous avons démontré la faisabilité d'intégrer ces composantes dans un outil unique, accessible et pédagogique. Les validations numériques ont confirmé la justesse des implémentations, et les analyses ont mis en évidence l'impact des différents modèles et paramètres sur le prix des options.